

Note Completo dai File C#

TIPI DI DATI IN C#

Tutti i tipi derivano da **System.Object** (il tipo padre di tutto!)

Interi Signed (Con Segno)

Tipo	Alias	Byte	Range
int	System.Int32	4	-2,147,483,648 a 2,147,483,647
short	System.Int16	2	-32,768 a 32,767
long	System.Int64	8	±9,223,372,036,854,775,807
sbyte	System.SByte	1	-128 a 127

Interi Unsigned (Senza Segno)

Tipo	Alias	Byte	Range
uint	System.UInt32	4	0 a 4,294,967,295
ushort	System.UInt16	2	0 a 65,535
ulong	System.UInt64	8	0 a 18,446,744,073,709,551,615
byte	System.Byte	1	0 a 255

Decimali e Floating Point

Tipo	Alias	Byte	Uso
float	System.Single	4	Precisione singola
double	System.Double	8	Precisione doppia (default)
decimal	System.Decimal	16	Valori monetari

			(alta precisione)
--	--	--	-------------------

Testo e Booleani

Tipo	Alias	Byte	Dettagli
string	System.String	variabile	Collezione di char (immutabile)
char	System.Char	2	Carattere singolo (UTF-16)
bool	System.Bool	1	true / false

Speciali

Tipo	Alias	Uso
void	System.Void	Nessun valore di ritorno
object	System.Object	Tipo generico (padre di tutti)
dynamic	System.Dynamic	Interfaccia con altri linguaggi

INPUT/OUTPUT

Lettura da Console

```
Console.Write("Testo: ");          // Senza newline
string input = Console.ReadLine(); // Legge una riga
```

Scrittura su Console

```
Console.Write("Testo");           // Senza newline
Console.WriteLine("Testo");       // Con newline
```

STRINGHE - Placeholder e Formattazione

1. Composite Formatting (Vecchio Stile)

```
Console.WriteLine("Hello, {0}. Today is {1}", name, "Thursday");  
// {0} = primo parametro  
// {1} = secondo parametro
```

2. Interpolated-String (Moderno - Preferito)

```
Console.WriteLine($"Hello, {name}. Welcome to C#");  
// Più leggibile e pulito
```

3. Concatenazione Classica

```
string result = "Hello, " + name + ". Welcome to C#";
```

4. Allineamento

```
int a = 100;  
Console.WriteLine($"Value: {a,-20}. Rest of text.");  
// -20 = allineamento a SINISTRA in 20 caratteri  
// +20 = allineamento a DESTRA in 20 caratteri
```

5. Formattazione Valori

```
int a = 100;  
Console.WriteLine($"Hex: {a:X}");           // 64 (esadecimale)  
Console.WriteLine($"Digits: {a:D5}");       // 00100 (5 cifre)  
  
// Dates  
DateTime dt = DateTime.Now;  
Console.WriteLine($"Date: {dt:D2}");        // 2 cifre con zero padding
```

DateTime

```

DateTime dt = DateTime.Now; // Data e ora corrente

// Proprietà
dt.Year      // Anno
dt.Month     // Mese
dt.Day       // Giorno
dt.Hour      // Ora
dt.Minute    // Minuti
dt.DayOfWeek // Giorno della settimana

// Inizializzazione default
var dt2 = new DateTime(); // 01/01/0001 00:00:00

```

INFINITE LOOP

```

for (;;) {
    // Codice che si ripete infinitamente
    if (condizione) break; // Necessario per uscire
}

// Equivalente a:
while (true) {
    if (condizione) break;
}

```

Uso Pratico: Validazione Input

```

string ReadName(string prompt) {
    for (;;) {
        Console.Write(prompt);
        string name = Console.ReadLine();
        if (!IsBlank(name)) return name; // Esce se valido
    }
}

bool IsBlank(string s) {
    for (int i = 0; i < s.Length; i++)
        if (s[i] != ' ') return false; // Trovato carattere non-spazio
    return true; // Sono solo spazi
}

```

PATTERN MATCHING con `is`

Vecchio Stile (Ripetitivo)

```
if (DateTime.Now.DayOfWeek == DayOfWeek.Saturday ||
    DateTime.Now.DayOfWeek == DayOfWeek.Sunday)
    weekEnd = true;
else
    weekEnd = false;
```

Modern Style con Logical Pattern

```
if (DateTime.Now.DayOfWeek is DayOfWeek.Saturday or DayOfWeek.Sunday)
    weekEnd = true;
else
    weekEnd = false;

// Ancora più compatto (Assignment)
bool weekEnd = DateTime.Now.DayOfWeek is DayOfWeek.Saturday or Day-
OfWeek.Sunday;
```

Vantaggi: Codice più compatto, non ripeti la variabile da confrontare

EXCEPTION HANDLING

Metodo 1: Try-Catch (Rimuove l'eccezione)

```
int ReadInt(string prompt) {
    for (;;) {
        Console.Write(prompt);
        string value = Console.ReadLine();
        try {
            return int.Parse(value); // Può lanciare eccezione
        } catch {
            Console.WriteLine("Enter a proper number, please!");
        }
    }
}
```

Problema: `Parse()` lancia eccezione se non è un numero valido

Metodo 2: TryParse (Ritorna bool - Preferito)

```

int ReadInt2(string prompt) {
    for (;;) {
        Console.Write(prompt);
        if (int.TryParse(Console.ReadLine(), out int result))
            return result; // Conversione riuscita
        else
            Console.WriteLine("Enter a proper number, please!");
    }
}

```

Vantaggi:

- Più efficiente (no eccezioni)
- Più leggibile
- `out int result` cattura il valore convertito

ESERCIZIO 1: Numero Segreto (User Indovina)

```

int numb = new Random().Next(1, 100);
Console.WriteLine("Find the number which i've chosen");
Console.WriteLine(CheckGuess(numb));

string CheckGuess(int numb) {
    string correct = "You guessed correctly, Good job!";
    string toolow = "Your guess is too low, Retry";
    string toohigh = "Your guess is too high, Retry";
    int tentativo = 1;

    for (;;) {
        int guess = ReadInt2("Enter a number: ");
        if (guess == numb) {
            Console.WriteLine($"Numero tentativi: {tentativo}");
            return correct;
        }
        else if (guess > numb) {
            Console.WriteLine(toohigh);
            tentativo++;
        } else {
            Console.WriteLine(toolow);
            tentativo++;
        }
    }
}

```

Concetti Usati:

- `Random().Next()` per numero casuale
- Loop infinito con `break`
- Contatore tentativi
- Comparazione valori

ESERCIZIO 2: Binary Search (PC Indovina)

Il PC indovina un numero che l'utente pensa usando **Binary Search Algorithm**!

```
int GuessYourNumber() {
    int min = 1;
    int max = 100;
    int numToFind;
    int iter = 1;

    for (;;) {
        numToFind = (min + max) / 2; // Midpoint

        Console.WriteLine($"Is the number: {numToFind}");
        Console.WriteLine("Write yes or no: ");
        string answ = Console.ReadLine().ToLower();

        if (answ == "yes") {
            Console.WriteLine("Good, i've found it!");
            Console.WriteLine($"Number of guesses: {iter}");
            return numToFind;
        }
        else if (answ == "no") {
            Console.WriteLine("Is the number too high or too low?");
            string answ2 = Console.ReadLine().ToLower();

            if (answ2 == "low") {
                min = numToFind + 1; // Numero è più alto
                iter++;
            }
            else if (answ2 == "high") {
                max = numToFind - 1; // Numero è più basso
                iter++;
            }
            else
                Console.WriteLine("Invalid input. Try again");
        }
        else
            Console.WriteLine("Invalid input. Try again");
    }
}
```

```
}  
}
```

Algoritmo Binary Search

Range: 1-100

Iterazione 1: Chiede 50 'Too high' Range: 1-49

Iterazione 2: Chiede 25 'Too low' Range: 26-49

Iterazione 3: Chiede 37 'Too low' Range: 38-49

...

Massimo 7 iterazioni per indovinare ($\log_2 100 \approx 7$)

Vantaggi:

- Velocissimo (logaritmico)
- Efficiente
- Dimostra logica di algoritmo

PAROLE CHIAVE

Keyword	Uso
new	Crea nuovo oggetto/istanza
out	Parametro di output (TryParse)
try-catch	Gestione eccezioni
for	Loop con contatore
while	Loop con condizione
if-else	Condizionale
return	Ritorna valore dalla funzione
break	Esce dal loop
is	Pattern matching
or	Operatore logico OR (pattern matching)

BEST PRACTICES

1. Preferisci **TryParse** a **Parse + Try-Catch**
2. Usa **Interpolated-String** invece di Composite Formatting
3. Usa **Pattern Matching** `is` per condizionali più puliti
4. Valida sempre l'input utente (infinite loop + condizione)
5. Conta i tentativi quando fai indovinare numeri
6. Usa **Binary Search** per ricerche efficienti
7. `ToLower()` per confronti case-insensitive

RIASSUNTO FLUSSO CODICE

```

START
“
READ INPUT (ValidateInput in infinite loop)
“
PROCESS (CheckGuess o GuessYourNumber)
“
IF CORRECT ' RETURN + OUTPUT

```

```
“  
ELSE ' UPDATE STATE + RETRY (loop)  
“  
END
```

Concetto chiave: Input validation + Infinite loop + Escape condition